

Ingeniería del Software de Componentes y Sistemas Distribuidos

PRUEBA DE EVALUACIÓN CONTINUADA – Práctica 3

Presentación

En la Práctica 2 se han implementado tres microservicios del caso **Photo&Film4You**. En esta Práctica 3 verificaremos el funcionamiento correcto de los microservicios mediante pruebas automatizadas.

Competencias

En esta Práctica se trabajan las siguientes competencias del Grado de Ingeniería Informática:

- Saber proponer y evaluar diferentes alternativas tecnológicas para resolver un problema concreto.
- Aplicación de las técnicas específicas de la Ingeniería del Software a las diferentes etapas del ciclo de vida de un proyecto.

Objetivos

Los objetivos de la Práctica 3 son:

- Reconocer y proponer una estrategia de pruebas para una arquitectura basada en microservicios
- Reconocer y utilizar la simulación de dependencias de clases en las pruebas unitarias
- Reconocer y utilizar el uso de pruebas para verificar el contrato entre microservicios
- Reconocer y diferenciar la cobertura de la calidad de las pruebas, usando pruebas de mutación

Descripción de la Práctica 3 a realizar

La Práctica 3 se compone de 4 ejercicios. En cada uno de ellos hay que **justificar** todas las decisiones.

Para todos los ejercicios debéis partir de la solución de la Práctica 2 publicada en: <https://github.com/orgs/UOC-SPRING-2026/repositories>.

Ejercicio 1 - Estrategia de pruebas (2 puntos)

El archivo `CourseApplicationTests.java` solo contiene una prueba que verifica que el contexto de Spring arranca. Teniendo en cuenta la arquitectura de microservicios de la aplicación:

- Define una estrategia de pruebas para esta aplicación, indicando qué niveles de prueba existen, qué tipos de pruebas son necesarios para cada nivel y qué aspectos concretos cubrirían (1,5 puntos)

Justifica en detalle las decisiones y aporta adicionalmente un diagrama resumen en la que se indique para cada nivel de pruebas qué tipos de pruebas se incluyen y qué librerías o frameworks de pruebas automáticas se proponen.

- Resume qué pruebas aplicarías al microservicio de Cursos en cada nivel y tipo definido en la estrategia (0,5 puntos).

Ejercicio 2 - Pruebas unitarias y reporting (2 puntos)

Apartado 1 (1 punto)

Escribe una prueba **unitaria** para `CourseServiceImpl.createCourse()` usando JUnit5, Mockito y AssertJ. La prueba debe verificar los siguientes escenarios:

- El instructor existe → el curso se crea con estado DRAFT
- El instructor no existe → se lanza `UserNotFoundException` y no se invoca el método `createCourse` del `CourseRepository`

Justifica qué dependencias necesitas mockear y por qué no puedes instanciar `CourseServiceImpl` directamente sin mocks.

Notas:

En las pruebas unitarias no hacemos uso de ninguna infraestructura ni dependencia real. No usamos comunicación HTTP, servidores, bases de datos, servicios externos u otros microservicios. Todo aspecto relacionado con dependencias debe simularse (con **Mockito**), excepto las clases de dominio (`Course`, `Enrollment`, ...).

Apartado 2 (1 punto)

Utilizando como base las pruebas realizadas en el apartado anterior, debes configurar el adaptador de Allure correspondiente a Junit5 (<https://allurereport.org/docs/junit5/>) para transformar los logs de consola en un informe visual en HTML.

En el informe de entrega PDF se debe incluir:

- Pasos realizados para integrar el proyecto con Allure, junto con fragmentos relevantes de los archivos de configuración y del código de pruebas si se han modificado.
- Descripción breve del proceso para generar el archivo HTML de resultados y de su contenido (Overview, Timeline), apoyada por capturas de pantalla como evidencia.

Entrega adicionalmente un ZIP que incluya los archivos HTML y ficheros relacionados generados con Allure.

Ejercicio 3 – Pruebas de contrato (3,5 puntos)

Agregar pruebas de contratos es clave para garantizar que los servicios y microservicios funcionen correctamente juntos.

El objetivo de este ejercicio es generar pruebas automáticas de contrato utilizando **Pact** para validar los contratos establecidos entre *CourseService* (como consumidor) y *MicroCredentialsService* (como productor).

- a) Escenario 1. Creación exitosa de microcredenciales (/microcredentials/1/create): 201 Created o 200 OK
- b) Escenario 2. Curso no encontrado al crear unas microcredenciales (/microcredentials/999/create): 402

Debéis realizar:

- La implementación de la prueba de Pact en el lado del consumidor donde se definen las interacciones para los dos escenarios solicitados (debe incluirse la configuración del PactBuilder y el uso del servidor mock).
- Generar con Pact el archivo JSON que contiene las expectativas del contrato
- La implementación de la prueba de Pact en el lado del proveedor.
- Informe de ejecución con capturas o logs:
 - Lado Consumidor: Evidencia de que las pruebas pasaron y el archivo JSON se generó correctamente.
 - Lado Proveedor: Evidencia de que el proveedor validó con éxito el contrato enviado por el consumidor.

Notas:

Podéis usar como referencia para ayudaros a desarrollar las pruebas:

- <https://www.baeldung.com/pact-junit-consumer-driven-contracts>

Ejercicio 4 – Pruebas de mutación (2,5 puntos)

Se ha implementado una nueva clase *CourseValidator* que contiene lógica de negocio para validar diferentes aspectos en la creación de los cursos en el sistema. Actualmente, incluye las siguientes validaciones o criterios de aceptación:

- Validación de precio: Rango permitido entre 0 y 10.000€
- Validación de duración: Entre 1 y 500 horas
- Cálculo de descuentos: Basado en rangos de precio

El código es el siguiente:

```
package edu.uoc.epcsd.course.domain.validator;

public class CourseValidator {

    private static final int MIN_PRICE = 0;
    private static final int MAX_PRICE = 10000;
    private static final int MIN_DURATION = 1;
    private static final int MAX_DURATION = 500;

    /**
     * Validates whether a course price is valid.
     * It must be between 0 and 10000.
     */
    public boolean isValidPrice(Long price) {
        if (price == null) {
            return false;
        }
        return price >= MIN_PRICE && price <= MAX_PRICE;
    }

    /**
     * Validates whether the course duration is valid.
     */
}
```

```

* It must be between 1 and 500 hours.
*/
public boolean isValidDuration(Long duration) {
    if (duration == null) {
        return false;
    }
    return duration >= MIN_DURATION && duration <= MAX_DURATION;
}

/**
 * Calculates the applicable discount based on the price.
 * - Price >= 5000: 20% discount
 * - Price >= 2000: 10% discount
 * - Otherwise: 5% discount
 */
public int calculateDiscount(Long price) {
    if (price == null || price < 0) {
        return 0;
    }
    if (price >= 5000) {
        return 20;
    } else if (price >= 2000) {
        return 10;
    } else {
        return 5;
    }
}
}

```

Así mismo se ha implementado una prueba unitaria que verifica estos criterios de aceptación:

```

package edu.uoc.epcsd.course.domain.validator;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.assertj.core.api.Assertions.assertThat;

```

```
class CourseValidatorTest {
    private CourseValidator validator;

    @BeforeEach
    void setUp() {
        validator = new CourseValidator();
    }

    @Test
    void testIsValidPrice() {
        assertThat(validator.isValidPrice(5000L)).isTrue();
        assertThat(validator.isValidPrice(null)).isFalse();
    }

    @Test
    void testIsValidDuration() {
        assertThat(validator.isValidDuration(100L)).isTrue();
        assertThat(validator.isValidDuration(null)).isFalse();
    }

    @Test
    void testCalculateDiscount() {
        assertThat(validator.calculateDiscount(7000L)).isEqualTo(20);
        assertThat(validator.calculateDiscount(3000L)).isEqualTo(10);
        assertThat(validator.calculateDiscount(1000L)).isEqualTo(5);
        assertThat(validator.calculateDiscount(null)).isEqualTo(0);
    }
}
```

En este ejercicio debéis realizar los siguientes pasos:

1. Revisa los tests (*testIsValidPrice*, *testIsValidDuration*, *testCalculateDiscount*). ¿Qué porcentaje de cobertura de líneas tiene la clase *CourseValidator*? Describe el significado de éste valor para esta clase.
2. Configura y ejecuta **piTest** (<https://pitest.org/>) en el proyecto. Explica cómo se configura piTest, qué archivos y fragmentos se modifican, y cómo generar el informe HTML de resultados.

3. Revisa el informe HTML de piTest y justifica las métricas siguientes asociadas a las clases *CourseValidatorTest* y *CourseValidator*:
 - a. Mutation coverage
 - b. Line Coverage
 - c. Desglose por mutante. Análisis de cada tipo de mutación
 - d. Mutaciones supervivientes. Cuántos mutantes han sido generados y cuántos de ellos han sobrevivido. Justificar por qué han sobrevivido.
4. Justifica qué pruebas son necesarias para “matar” a los mutantes que han sobrevivido e implementarlas.
5. Re-ejecuta piTest y aporta el nuevo resultado HTML en el que se demuestre que se han eliminado todos los mutantes supervivientes.

Notas:

El código mostrado en el enunciado no está en la solución de la PRAC2, debéis copiar su contenido en vuestro repositorio, dentro del directorio src/test.

Recursos

Recursos Básicos

- Libro "Microservices Patterns" (Chris Richardson): Chapter 9. "Testing microservices: Parte 1". Introduction.
- Libro "Microservices Patterns" (Chris Richardson): Chapter 10. "Testing microservices: Parte 2".
- Libro "Fundamentals of Software Architecture" (Richards & Ford).
 - o Chapter 3. "Modularity".
 - o Chapter 6. "Measuring and Governing Architecture Characteristics".

Criterios de evaluación

- En esta actividad **no está permitido el uso de herramientas de inteligencia artificial** y se tiene que resolver de forma **estrictamente individual**. En caso de detectar irregularidades se penalizará la actividad con una D como nota. Esto incluye la reutilización de soluciones de esta práctica de semestres anteriores. En el plan docente y en el [web sobre integridad académica y plagio de la UOC](#) encontraréis información sobre qué se considera conducta irregular en la evaluación y las consecuencias que puede tener.
- El peso de cada ejercicio está indicado en el enunciado.
- Los criterios de evaluación de cada ejercicio están indicados en el anexo.

- Es necesario justificar la solución a cada uno de los ejercicios. Se valorará tanto la corrección de la solución como la justificación dada.

Formato y fecha de entrega

Hay que entregar un solo documento PDF llamado *ApellidosNombre_ISCSDPRAC3.pdf*, donde se explique cómo se ha llevado a cabo la práctica, justificando las decisiones tomadas y proporcionando instrucciones para ejecutar las pruebas.

Además, se debe adjuntar por separado:

1. El código de las pruebas en un archivo ZIP
2. Los resultados HTML en Allure del segundo ejercicio, en otro archivo ZIP independiente: *Ejercicio2.zip*.
3. Los resultados de las pruebas del tercer ejercicio, en otro archivo ZIP independiente: *Ejercicio3.zip*.
4. Los resultados HTML de piTest del cuarto ejercicio, en otro archivo ZIP independiente: *Ejercicio4.zip*.

El documento PDF y los 2 ficheros ZIP se entregarán en el espacio “Entrega de la PRAC3” del aula antes de las **23:59 horas del día 8 de junio de 2026**. No se aceptarán entregas fuera de plazo.

Anexo

Criterios de evaluación de la Práctica 3

	Excelente (10 puntos)	Notable (7 puntos)	Suficiente (5 puntos)	Insuficiente (2,5 puntos)	Sin respuesta / Todo mal (0 puntos)
Ej. 1	Niveles y tipos de prueba definidos con detalle. Se justifica exhaustivamente cada decisión y el diagrama resumen es coherente con la arquitectura de microservicios	Se definen los niveles correctamente, pero la justificación o la selección de librerías es mejorable. Se incluye el diagrama solicitado.	Se describen los niveles de forma genérica. La justificación se presenta de forma básica y el diagrama carece de detalles o de alguna librería clave	No se identifican correctamente los niveles de prueba o se omite el diagrama resumen y su aplicación al microservicio de Cursos	No se ha contestado o el contenido no tiene relación con estrategias de prueba para microservicios
Ej. 2.1	Se implementan los escenarios de creación y error mediante JUnit5, Mockito y AssertJ. Se justifica técnicamente por qué se requieren mocks y no instanciación directa.	Se implementan los escenarios, pero se omiten detalles en las aserciones o la justificación del uso de mocks se presenta de forma breve.	Se presenta una prueba funcional, pero se hace un uso incorrecto de AssertJ o la justificación sobre dependencias es incompleta.	Se implementa solo un escenario o se utiliza infraestructura real (DB/HTTP), sin usar mocking	No se ha entregado el código o este no compila ni se hace uso de la simulación de dependencias
Ej. 2.2	Se documentan detalladamente los pasos de integración. Se incluyen capturas claras de Overview y Timeline, y se ha entregado el archivo ZIP funcional.	Se realiza la integración correctamente, pero se omiten algunas capturas o la descripción en el PDF se considera poco detallada.	Se genera el informe visual, pero la documentación aportada se muestra escasa. Se entrega el ZIP con errores menores.	No se integra el adaptador de Allure correctamente o no se adjunta el ZIP con los resultados.	No se evidencia la integración con Allure o no se ha entregado el informe HTML en el ZIP.

Ej. 3	Se implementan los dos escenarios en consumidor y proveedor. Se genera el archivo JSON y se aportan evidencias de ejecución de ambos lados.	Se implementan los escenarios, pero las evidencias se muestran confusas o la configuración del PactBuilder es mejorable	Se implementa sólo un escenario o se omiten las evidencias de validación en el lado del proveedor.	No se entrega el archivo JSON o el proveedor no valida el contrato. Se detectan errores graves en la lógica.	No se implementan pruebas de contrato o se utilizan mocks simples sin la librería Pact.
Ej. 4	Se analizan las métricas de cobertura y mutantes. Se implementan pruebas adicionales que matan a los mutantes supervivientes, demostrado en el informe y en el ZIP.	Se configura piTest y se identifican mutantes, pero la justificación de su supervivencia no es del todo correcta. Se eliminan la mayoría de mutantes.	Se ejecuta piTest y se entregan informes , pero la implementación de nuevos tests se muestra insuficiente para mejorar la calidad	No se justifican las métricas solicitadas o no se aportan los nuevos resultados tras intentar matar mutantes.	No se entregan los informes de piTest ni se realiza el análisis de la clase CourseValidator.